Bitmor: A BTC-Backed Undercollateralised Lending & Borrowing Protocol

Version: Draft v0.2
Date: 12 Jan 2026

Abstract

Bitmor enables users to acquire BTC with a down payment and predictable monthly repayments, built atop an Aave-style pooled credit engine with new mechanisms for scheduled amortization, micro-liquidations, per-loan insurance and yield on the loaned BTC.

Contents

---

1. Introduction

Bitmor is an undercollateralised borrowing protocol, where user supplies USDC and borrow cbBTC which is locked on a per-loan basis. The protocol provides fixed, predictable *scheduled* monthly payments while interest accrues via standard utilization-based variable interest rates and index-based accounting.

Pool-based credit aligns with established DeFi primitives: (i) deposit tokens that accrue yield, (ii) utilization-driven interest rate models (IRM) with a kink, and (iii) scaled balance accounting via liquidity and borrow indices. Bitmor adopts those components and adds loan-level mechanics specific to financing a BTC purchase with amortization and robust default handling.

A lot of functionalities in the protocol are inspired by Aave. The IRM, aToken-style deposit representation, index-based accrual with lazy (“accrue-on-action”) updates, and the ordering of liquidation proceeds.

However, Bitmor introduces innovations aimed at providing a tool for BTC acquisition**.**

(i) USDC is borrowed via flash loan, auto-routed to buy BTC;

(ii) The BTC is deposited into the Bitmor Lending pool, ISDC is borrowed against this collateral and the flash loan is paid back.

(iii) Each loan has a conservative, fixed monthly payment schedule sized at origination;

(iv) Missed payments trigger micro-liquidations that only sell what is needed to stay current;

(v) Per-loan protective put-option suppresses price-based liquidations while active.

(vi) The BTC purchased on the loan can be deposited into whitelisted protocols to earn yield.

---

2. Protocol Architecture

Bitmor's architecture mirrors a proven pool layout with loan level liquidations, monthly repayments and insurance specific extensions:

2.1 Lending Pool (Interface)

The Lending Pool interacts with the rest of the system using *User operations.* Supply/withdraw USDC (mint/burn interest-bearing aTokens), borrow (BTC purchase is atomic), repay monthly amounts or preclose loans, and trigger liquidations where permitted. This also holds the liquidation mechanics and loan level information.

2.2 BTC Yield Vault

The BTC purchased is deposited into a ERC4626 compatibleYield generation vault. This Vault can curate yields between various whitelisted protocols for the BTC present in the system.

2.3 Loan Provider and Loan Account

The loan provider is the core contract that helps in the initiation of a loan and interacting with all other periphery contracts. Loan Account is the individual per loan contract where the tokens are held. It holds the Debt and the assets of the loan.

2.4 Insurance Manager

Insurance manager calculates the amount and Strike of PUT Option that need to be bought to protect lenders capital and the borrower from price movement based liquidations. It also manages purchase and sale of these Options at appropriate times.

2.5 Libraries and managers

Libraries are a concept borrowed from Aave. They reduce the gas footprint of all the actions by 15 to 20% while optimising the code complexity and verbosity.

2.6 Debt tokens

The variable debt token follows the scaled balance approach. When a user borrows or repays their loans, their scaled balances of tokens update using indices.

2.7 USDC Vault

The liquidity providers will deposit funds in the USDC Vault which will allocate funds to the Bitmor Lending Pool as per the requirements and utilize remaining funds to generate yield through battle tested protocols like Aave.

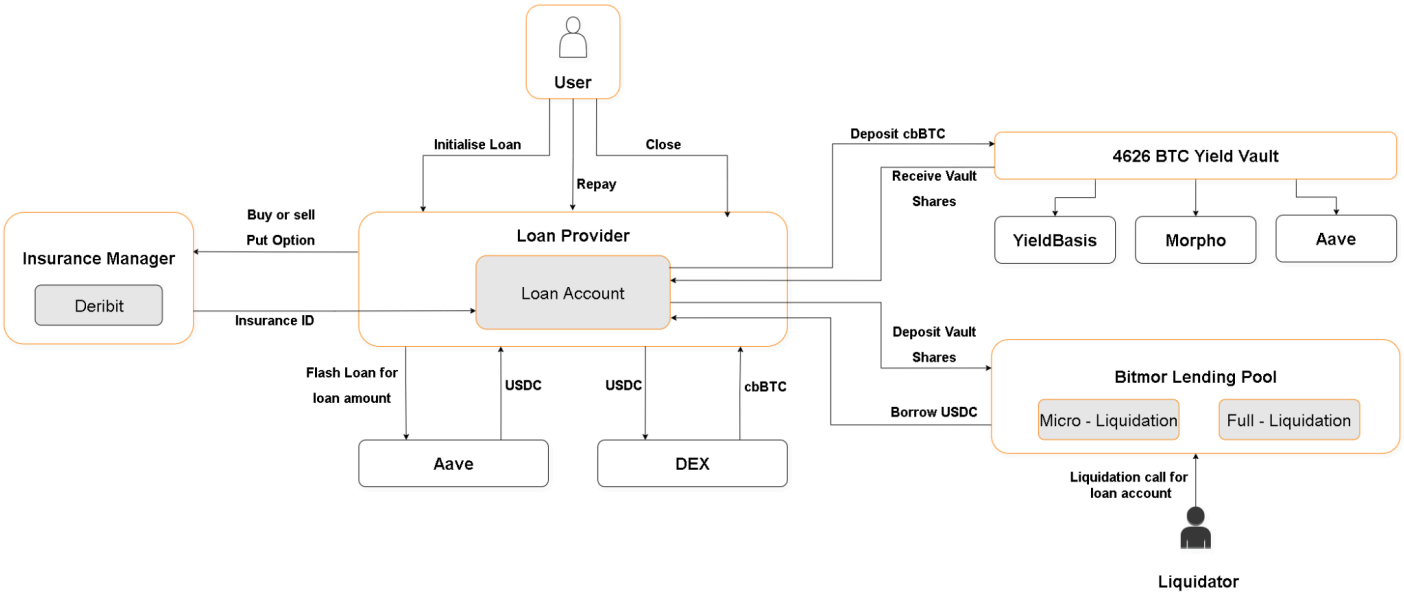


Figure 1: Protocol Architecture

---

3. Monthly Repayments

Each loan is created with a *scheduled monthly payment* computed at origination using the **maximum possible borrow APR** implied by the IRM. Accrual continues at the live variable rate via indices; the schedule provides predictable cash-flow and a conservative bound.

Notation.

- `P` — principal (USDC borrowed)
- `n` — number of months in term
- `r_max_annual = base_apr + slope1_apr + slope2_apr`
- `r = r_max_annual / 12` (conservative monthly rate for sizing only)
- `A` — fixed scheduled monthly amount (annuity)

The monthly payment sizing or annuity is calculated as follows:

$A = P * r(1 + r)^n/((1 + r)^n - 1)$;

$r = 0 => A = P/n$;

Note: this r is not the live accrual rate; it is only a conservative sizing assumption to keep the monthly amount predictable. Actual interest accrues at the current variable APR via the indices.

Accrual indices - These indices are the Aave inspired method to keep track of the interest accrued for both lenders and borrowers. Debt grows by the variable borrow index; deposits grow by the liquidity index. Between actions, front-ends may project "virtual indices" for live display, but on-chain state updates lazily on interaction.

Collateral release rule - Once full repayment is complete, only then the BTC becomes withdrawable.

Store in Loan State - Each loan has its monthly due date, next due timestamp, last due timestamp, partial paid amount and duration recorded in the Loan State.

Monthly payment wrapper - This is a helper which previews the next month's pay, it is called pay_one. It is simply the scheduled monthly amount, but never more than the current remaining debt. This is the value that is displayed to the user on their due date to be paid every month.

$pay\_one = min(A, debt\_remaining)$

Monthly payment: When a monthly payment is made, we reduce scaled debt by $scaled\_delta = min(pay / idx, scaled\_debt)$

Rebase $last\_settled\_index$:

If accrued interest remains:

$set\ to\ idx - (remaining\_interest / scaled\_debt)$;

Else:

$set\ to\ idx$

Pool liquidity: When a repayment is made, we increase **available_liquidity** by **pay** (the total amount paid) irrespective of its interest/principal split.

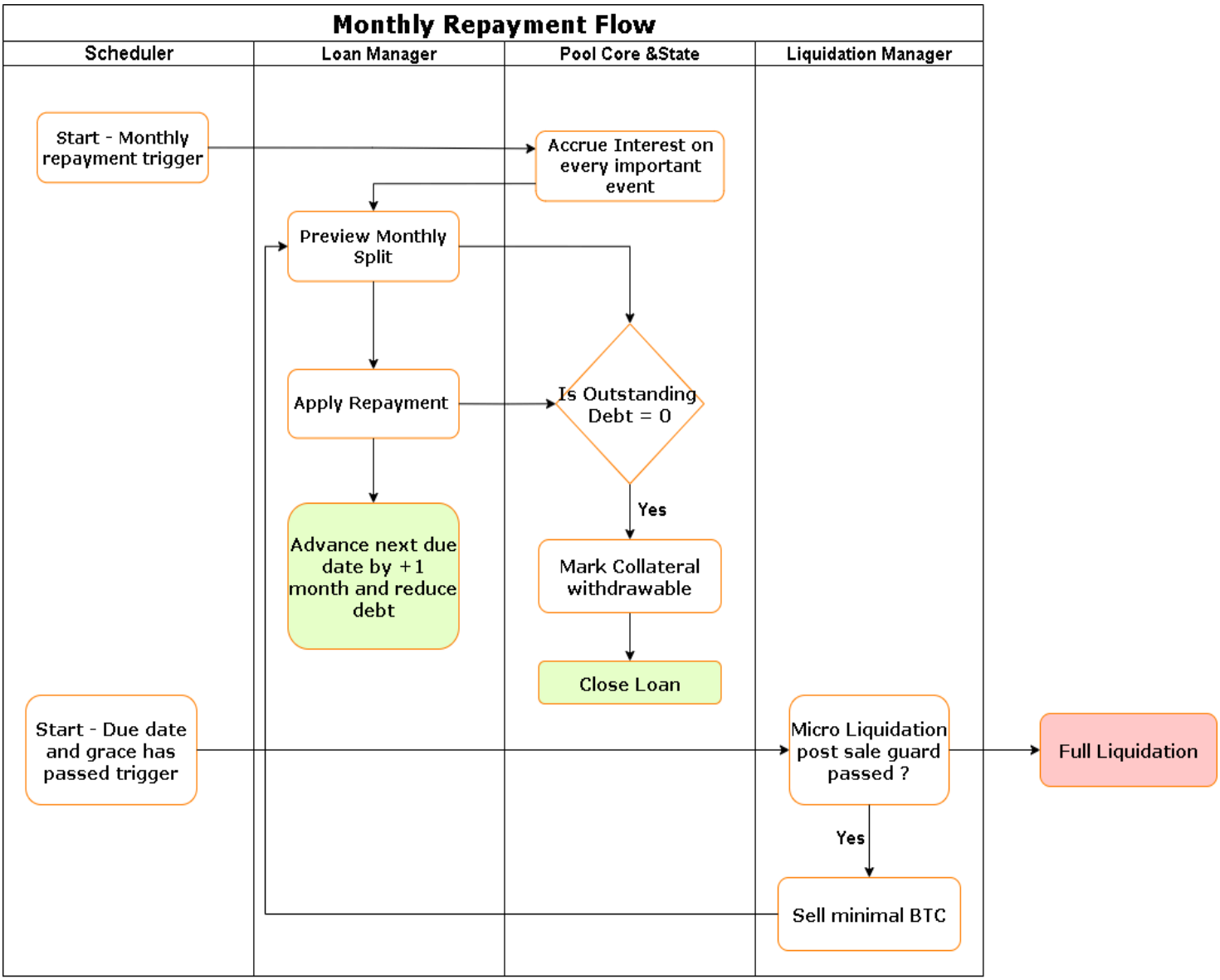


Figure 2: Monthly Repayment Flow

---

4. Liquidations & Micro-Liquidations

The following are two main reasons why a liquidation or micro-liquidation would be triggered in Bitmor.

4.1 Full Liquidation Triggers (per-loan)

- **Price-based (uninsured loans only):** if allocated BTC value drops below debt by a governance-set buffer, the user is fully liquidated.
- **Non-payment:** if a scheduled payment is missed and grace period also expires, protocol attempts a *micro-liquidation* that sells only enough BTC to fund one monthly payment (+ liquidator fee); it might also escalate to full liquidation if post-sale safety fails.

NOTE: Insured loans are not triggered for price-based liquidation.

Key quantities.

- $allocated\_BTC = max(0, BTC\_bought - Total\_BTC\_sold)$
- $debt\_now = scaled\_debt \times variable\_borrow\_index$
- $price$ — $oracle\ BTC\ price$;
- buf — $liquidation\ buffer\ (e.g., 3\%)$

Price-based full-liquidation trigger (if uninsured):

$$Trigger = allocated \times price < debt\_now \times (1 + buf)$$

If the above trigger is active for an uninsured borrower, liquidators are free to retrieve the collateral in the loan and execute a full liquidation.

Allocation of proceeds in full liquidation: First we calculate proceeds from BTC (and, for non-payment, sell the PUT option based insurance), repay the liquidator with BTC + proceeds from the PUT option enough to cover debt and fees;.

Liquidator incentive (full):

$$bonus\_full = buf \times debt\_now$$

4.2 Micro-Liquidation (missed-payment guardrail)

In the case of a missed payment, a micro-liquidation is enabled. By selling the minimal BTC required while also preserving loan health a user can continue to own a profitable position without being liquidated.

Value of one monthly payment:

This is stored as a value called pay_one . It is simply the next scheduled monthly payment, capped by what's actually outstanding. The user should never have to pay more than what is currently outstanding.

Sizing the cash needed to micro-liquidate one monthly payment:

$$cash\_needed = pay\_one + buf \times pay\_one$$

BTC sold to cover the monthly payment:

$$sold\_btc = cash\_needed / price$$

Post-sale guard: after selling just enough BTC to raise `cash_needed`, we require the remaining BTC value to respect the buffer against the *post-payment* full remaining debt:

$$Remaining\_BTC\_value = (allocated\_BTC - sold\_btc) * price$$

$$Remaining\_BTC\_value \geq D\_after + buf \times debt\_now$$

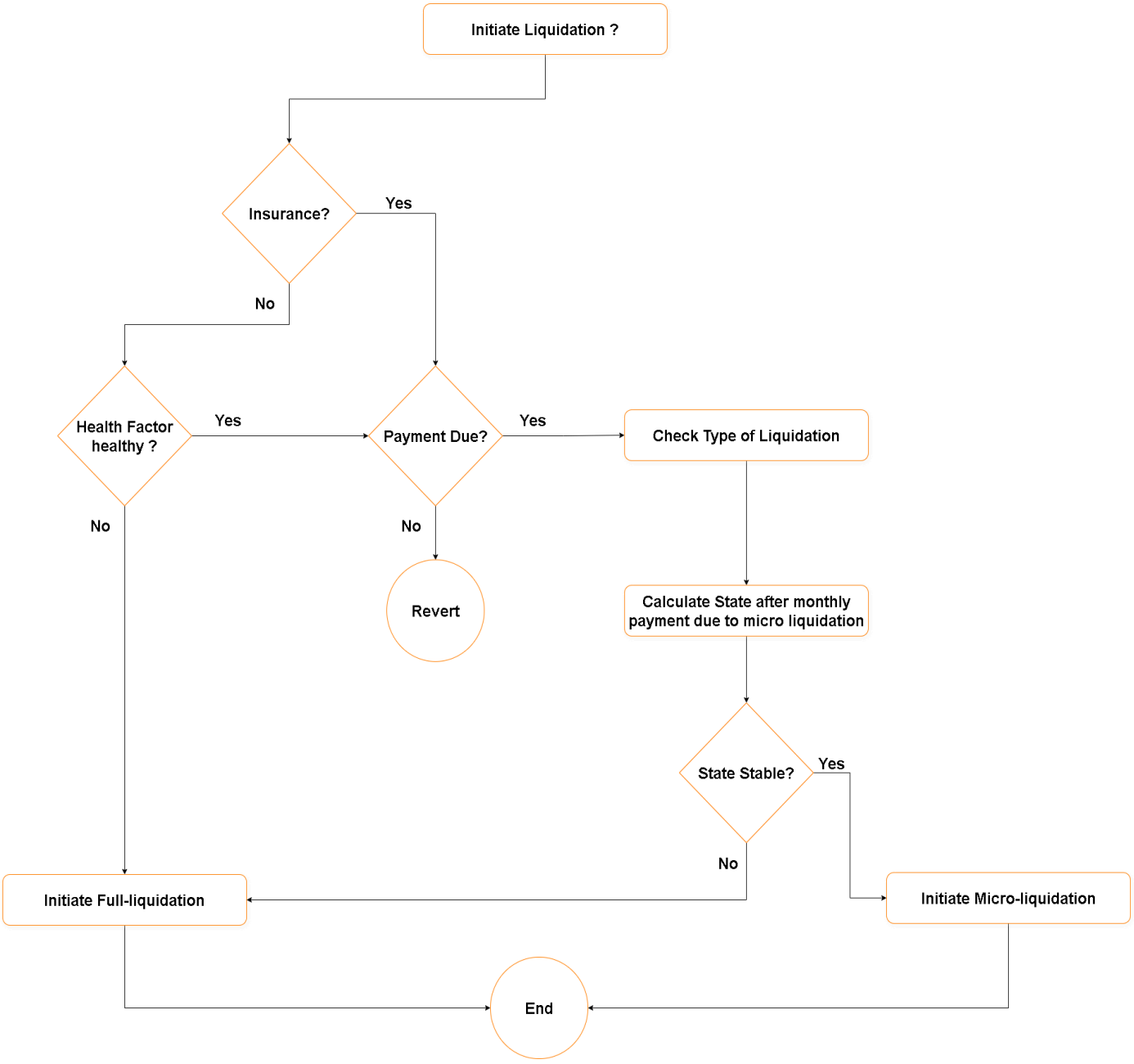
$$Where,\ D\_after = debt\_now - pay\_one$$

If either condition fails (insufficient collateral value or guard violation), the protocol escalates to full liquidation.

Liquidator incentive (micro):

$$bonus\_micro \;=\; buf \times pay\_one$$

Note: Liquidations are per-loan. Proceed ordering is deterministic: pool → liquidator.



5. Insurance

Bitmor offers an optional per-loan BTC **PUT option** cover, sized to the BTC bought for that loan. While cover exists, **price-based** liquidation is disabled. However, even insurance does not protect users from non-payment liquidations.

In those, BTC is sold first; and remaining puts are sold to cover the shortfall such that the lender's capital and liquidators fee is always protected.

Position model. A per-loan FIFO list of `PutOption at (strike K, qty Q) is purchased`;

Where,

$K = price \times loan\_amount/(loan\_amount + deposit) \times 110\%$

$Q = collateral\_btc\_bought = (loan\_amount + deposit) \,/\, BTC\_price\_at\_purchase$

Coverage is trimmed FIFO whenever loan BTC is sold.

Liquidation conditions for insurance to be sold:

- **Blocks price-based liquidations.** If cover is active, only non-payment paths are allowed.
- **Non-payment path.** Sell BTC and the PUT option which was bought at 10% above the required strike price to cover the lender's principal, interest and the liquidator fee.
- After all proceeds are collected. Protocol returns the lender's principal and interest, pays the liquidator fee and returns any and all leftovers to the borrower.

---

6. Lending USDC

Lenders can deposit USDC into Bitmor and receive aUSDC tokens in return. These aUSDC tokens are rebasing assets that accrue interest generated by borrowers who take out USDC loans to purchase BTC.

Interest earnings follow a **two-slope utilisation-curve model**. The first slope increases the interest rate from a minimum of 5% at 0% utilisation to 10% at 80% utilisation. Beyond this point, the rate rises more sharply, reaching 15% at full utilisation. This structure encourages lenders to supply additional liquidity when the pool is running low.

When lenders first deposit USDC and no funds are currently borrowed, the idle liquidity is allocated to Aave V3's USDC pool on Base to ensure it continues to earn yield. However, the utilisation of the Bitmor pool is calculated using the total available liquidity so borrowers are not charged excessive interest simply because a portion of the liquidity sits in Aave.

Accordingly, Bitmor uses a **Virtual Utilisation** metric:

$$Virtual\ Utilisation\ =\ Total\ Borrowed\ /\ (Bitmor\ Balance\ +\ Aave\ Position)$$

This virtual utilisation determines the interest rates, allowing the system to maintain appropriate incentives for both lenders and borrowers.

The Vault also takes care that there is sufficient liquidity in the protocol for incoming borrowers, if liquidity lacks, it lets Bitmor pull liquidity from Aave and always prioritizes borrowers.

The USDC can be withdrawn by lenders based on availability. As loans grow, we expect incoming monthly payments to naturally feed into liquidity.

7. Conclusion

Bitmor merges a battle-tested pooled-credit core with three targeted innovations required for consumer-grade BTC financing: predictable scheduled repayments, micro-liquidations that minimize forced selling, and per-loan insurance that suppresses price-driven liquidations. The result is a protocol that keeps the capital-efficiency and transparency of DeFi, while delivering mortgages-like UX for Bitcoin acquisition.